THE ARCHITECT'S FIELD NOTES

BEHAVIORAL INTERVIEW STUDY GUIDE / 01 OF 01

AI Design #73 — Unified Enterprise Agentic Platform

THINK CLEARLY. DECIDE WITH EVIDENCE. DELIVER WITH CONFIDENCE.

Evidence-based STAR answers, senior-level judgment, and focused interview rehearsal.

PREPARED FOR SENIOR / PRINCIPAL SOLUTION ARCHITECT INTERVIEWS
READ · REFLECT · PRACTICE

01

YOUR STUDY COMPANION
Stage 2 of 6 — Essential Concepts

Capstone: Enterprise IT Operations Assistant

We will continue with incident `INC-42`, where the Checkout API became slow after a deployment.

This stage explains only the concepts needed to build our first working assistant. We will use simple examples, solid-line ASCII diagrams, and no full production architecture.

STAGE 2 — Essential AI architecture concepts

The following table summarizes the concepts we will introduce in this stage.

Concept	Simple definition	Applied to INC-42
LLM	Generates explanations or structured text from itsinput.	Summarizes why latency may have increased; does not know live metrics without supplied evidence.
Fixed workflow	Application code determines the order of steps.	Always read metrics and deployments, then analyze results.
Bounded agent	Model helps choose from permitted tools under execution limits.	Can propose which diagnostic read to make; cannot invent permissions.
Tool calling	Model proposes a typed operation; application validates and runs it.	`get_metrics` and `get_deployments` are approved reads.
Context engineering	Select information and instructions relevant to the current model call.	Give incident ID, approved evidence, source IDs, and uncertainty rules.
Grounding	Base material claims on retrieved records.	“Latency is 1,800 ms” cites `METRIC-07`.
RAG	Retrieve relevant knowledge passages before generation.	Useful later for large runbook or historical incident collections; unnecessary for two direct structured reads.
Control plane	Governance/configuration of capabilities.	Approved tools, tool versions, identity policy and budgets.
Runtime/data plane	Actual work performed on requests.	Incident workflow, authorized tool reads, and report generation.
Observability	Explains what executed, where, and with what latency/errors.	Shows tool timeline, failures and run correlation.
Evaluation	Tests whether the output is correct and useful.	Checks claims, citations, uncertainty and recommendation safety.

NAVIGATION

Inside this guide

01 1. Our starting problem

02 2. LLM versus AI agent

03 3. Tool calling — how AI accesses operational data

04 4. Context engineering and grounding

05 5. Control plane versus data plane

06 6. Observability versus evaluation

07 7. Which technologies do we actually need now?

08 Stage 2 summary

09 Next — Stage 3: Minimal Working Architecture

CHAPTER

1. Our starting problem

The on-call engineer asks:

> Investigate INC-42. Why is Checkout API slow, and what should I do next?

We already have two pieces of synthetic evidence:

Evidence	Observation
`METRIC-07`	p95 latency increased from 240 ms to 1,800 ms
`CHANGE-03`	Version `v2.7.4` deployed shortly before the slowdown

A normal application can retrieve these facts. But we want an assistant that can interpret them, explain possible causes, and recommend further investigation.

That distinction introduces our first concept.

CHAPTER

2. LLM versus AI agent

What is an LLM?

A **Large Language Model (LLM)** processes input and generates output, such as explanations, summaries, or structured text.

For example, provide the following evidence:

```
Checkout API latency: 1,800 ms
Normal latency: 240 ms
Recent deployment: v2.7.4
```

The model might respond:

> Checkout API latency is significantly higher than normal. The recent deployment may be related, but further evidence is needed.

It can explain metrics or summarize a deployment record, but it does not automatically know the current state of Checkout API.

It cannot investigate any system itself.

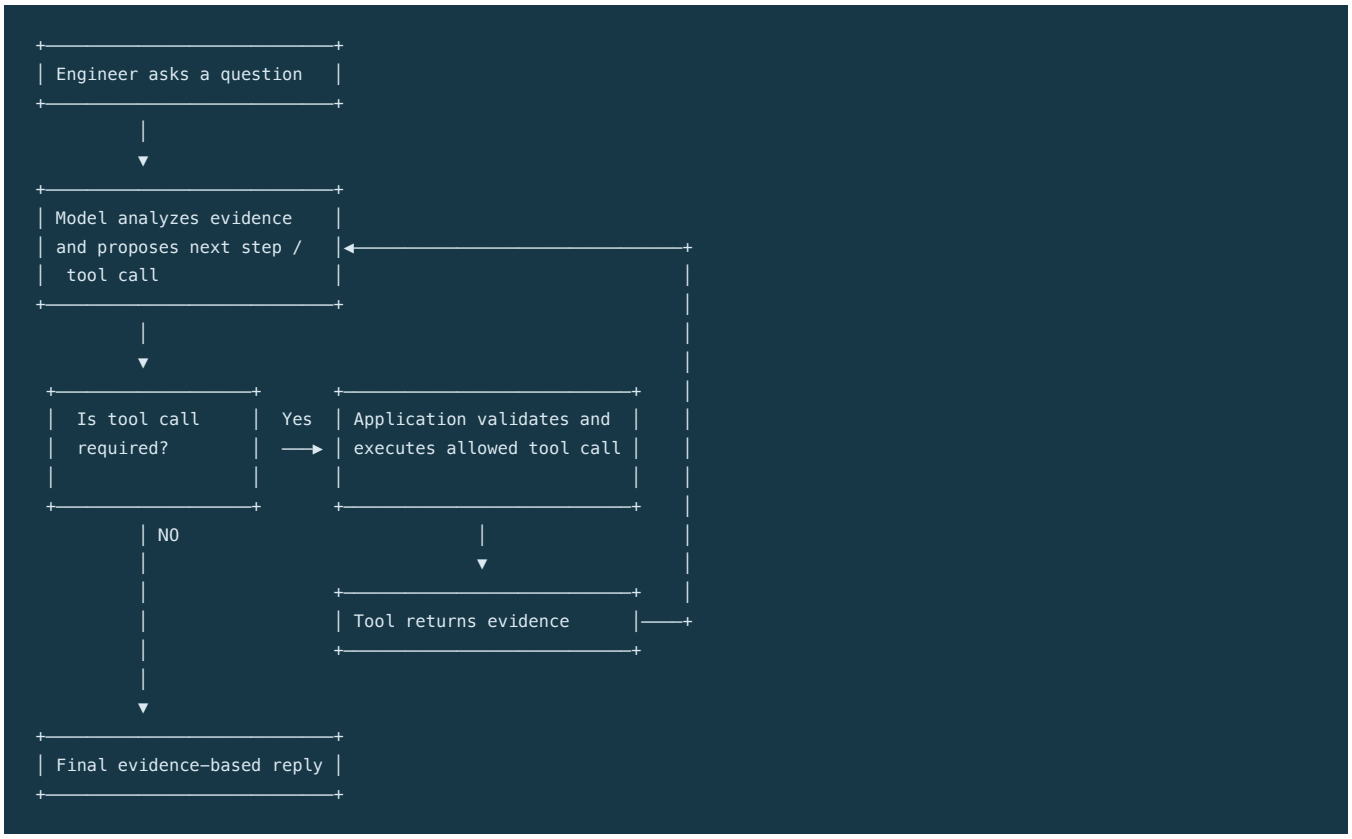
What is an AI agent?

An **AI agent** is an application that lets a model participate in deciding which tools or steps to request while pursuing a goal.

The application still executes and authorizes those steps.

For example, the agent may decide to inspect deployment history after detecting a latency spike.

ILLUSTRATIVE AGENT LOOP



The loop must have tool limits and a stopping condition. The model does not directly control production systems.

DECISION A: Do we need an agent immediately?

Not necessarily.

Our first investigation has two known data sources and a predictable sequence.

A fixed workflow can fetch those records before passing the evidence to an LLM.

Approach	Best when
Single LLM call	All necessary evidence is already provided
Fixed workflow + LLM	Required evidence sources and steps are known
Bounded agent	The next diagnostic step depends on earlier findings

Initial choice: Start with a fixed, read-only investigation workflow. Introduce adaptive agent behavior when the incident requires it.

This avoids unnecessary model decisions and makes failures easier to understand.

CHAPTER

3. Tool calling — how AI accesses operational data

An LLM does not automatically know the current state of Checkout API. It needs a way to obtain data from the systems that own that information.

A **tool** is an approved operation that the application can execute.

For our incident, we need two read-only tools:

Tool	Purpose
`get_metrics`	Retrieve latency and error measurements
`get_deployments`	Retrieve recent deployments

Example: tool request and response

The following is a conceptual tool request, not a specific SDK format.

TOOL REQUEST

```
{
  "tool": "get_metrics",
  "arguments": {
    "service": "checkout-api",
    "incident_id": "INC-42"
  }
}
```

TOOL RESPONSE — SYNTHETIC DATA

```
{
  "evidence_id": "METRIC-07",
  "service": "checkout-api",
  "p95_latency_ms": 1800,
  "baseline_p95_ms": 240,
  "error_rate_percent": 3.1
}
```

The application executes the tool against an approved data source. The model can then interpret the returned evidence.

Critical distinction



The model can request a tool call. The application decides whether that call is allowed.

Without this boundary, model-generated arguments could lead to unauthorized data access.

3.1 Which execution pattern fits?

Fixed workflow — Stage 3/4

Use when required sources and steps are known. Lower complexity and easier testing.

```
Request
  |
  ▼
Read metrics
  |
  ▼
Read deployments
  |
  ▼
Analyze evidence
```

Bounded tool selection — Stage 5A

Use when next reads may vary. The model proposes tools, but the controller enforces limits.

```
Request
  |
  ▼
Model proposes reads
  |
  ▼
Validate + authorize
  |
  ▼
Execute approved reads
  |
  ▼
Analyze + validate
```

CHAPTER

4. Context engineering and grounding

These concepts solve two related problems.

Context engineering means selecting the information that the model receives for its current task.

Grounding means making important claims depend on actual evidence rather than unsupported model output.

For INC-42, context might contain:

```
Incident: INC-42
Service: checkout-api

Evidence:
  METRIC-07: p95 latency = 1,800 ms
```

```
    CHANGE-03: deployment v2.7.4 at 09:55

Rules:
  Distinguish observations from hypotheses.
  Cite evidence IDs.
  Do not claim root cause without proof.
  Do not execute remediation.
```

4.1 Facts, hypotheses and missing information

The model should produce something like:

Type	Example	Evidence rule
Observed fact	Latency increased significantly(p95 latency is 1,800 ms).	Cite `METRIC-07`.
Observed fact	A deployment `v2.7.4` before degradation(at 09:55).	Cite `CHANGE-03`.
Hypothesis	The deployment may have contributed to degradation.	Use both observations; say unverified.
Unknown	Whether database saturation caused slow requests.	Do not manufacture database metrics.

Without grounding, the model might confidently declare the deployment responsible, even though another issue caused the slowdown.

Also, evidence retrieved from logs or documents must be treated as data, not instructions. A log entry cannot authorize the assistant to change production.

DECISION B: Do we need RAG?

Not yet.

Retrieval-Augmented Generation (RAG) becomes useful when the assistant needs to search a large collection of runbooks, incident reports or engineering documents.

For our first version, the data is structured and comes from two direct tool calls. A vector database is unnecessary.

CHAPTER

5. Control plane versus data plane

This distinction becomes especially important when we evolve the assistant into an enterprise platform.

A **control plane** governs how capabilities are configured and used.

A **data plane**, or **runtime plane**, carries out actual requests and tool operations.

TWO DIFFERENT RESPONSIBILITIES

```
+-------------------------------+
|         CONTROL PLANE         |
|                               |
| Approved tools and policies   |
| Configuration and ownership   |
+-------------------------------+
                |
                | Govern
```

```
              v
+-------------------------------+
|       RUNTIME / DATA PLANE    |
|                               |
| Investigate incident INC-42   |
| Execute authorized tool reads|
| Produce investigation result  |
+-------------------------------+
```

For example:

- **Control plane** defines that the Checkout API assistant may use approved read-only monitoring tools.
- **Data plane** executes the permitted metrics request when the engineer investigates `INC-42`.

Important: Central governance does not replace authorization in the business system. The monitoring service must still enforce its own access rules.

For our local mock, we do not need two independently deployed planes. We only need to understand the separation of responsibilities.

CHAPTER

6. Observability versus evaluation

An enterprise assistant must be both operationally reliable and useful.

These are different concerns.

Concept	Question	INC-42 example
Observability	What happened during execution?	Which tool was called? Did it fail? How long did it take?
Evaluation	Did the assistant produce a good result?	Did it cite the right evidence? Did it invent a root cause?

A request can complete successfully at the software level and still produce a bad investigation.

For example, all tools might return successfully, but the assistant could recommend restarting the wrong service.

We will eventually measure `task success`, `grounding accuracy`, `latency`, `tool errors` and `cost`.

For now, recording tool outputs and checking that every cited evidence ID exists is enough.

This distinction is part of the #73 reference's Stage 2 foundation.

CHAPTER

7. Which technologies do we actually need now?

Technology	Initial decision	Reason
LLM	Use	Explain findings and form hypotheses
Tool calling	Use	Obtain operational evidence
Fixed workflow	Use	Two known, read-only data sources

Context engineering	Use	Provide relevant evidence and instructions
MCP	Defer	Direct functions are enough for one local application
RAG	Defer	No large document collection yet
Durable orchestration	Defer	No long-running approval or write process yet
Generative UI / AG-UI / A2UI	Defer	A basic result view is sufficient
Multi-agent / A2A	Defer	No independent specialist agent is required
Enterprise controls	Start small	Enforce authorized reads and no production writes

MCP becomes valuable when tools/resources need a reusable host-to-server interface. It can expose reusable tools to compatible AI clients.

A2A is useful for delegating work between independently deployed/operating agents.

AG-UI streams agent state to owned frontends.

A2UI specifies trusted components declaratively.

MCP Apps embeds UI in a compatible host.

These are separate boundaries, not substitutes for one another. None are required just because an application uses an LLM.

We will revisit them when a specific requirement makes the additional complexity worthwhile.

CHAPTER

Stage 2 summary

Our initial approach is now clearer:

An authorized, read-only application retrieves facts. An LLM interprets those facts. The engineer remains responsible for remediation decisions.

We established the main conceptual boundaries:

Responsibility	Owner
User access and tool permissions	Application and business systems
Metrics and deployment facts	Operational source systems
Explanation and hypotheses	LLM
Operational decision	Authorized human
Governance of reusable capabilities	Future shared platform

CHAPTER

Next — Stage 3: Minimal Working Architecture

Stage 3 will turn these concepts into a concrete design. We will draw the smallest component diagram, specify the two tool contracts, trace one request from the engineer to the final response, and demonstrate an authorization or tool-failure path.

We will still avoid the full enterprise architecture.

Continue to Stage 3 Go deeper into Stage 2